

PROJECT EXECUTABLE FILES

Project Name: COMICRAFT

Date: 30 September 2026

Team ID: [ENTER TEAM ID]

Project Name: Comicraft

1. PROJECT EXECUTABLE FILES

Comicraft is a web-based comic creation and management application designed to help users create, customize, and manage digital comics in an easy and interactive manner.

The project source code, configuration files, dependency files, database-related files, and other required executable/deployable resources are organized and provided as part of the project submission.

The submitted files enable the evaluator to set up, execute, test, and demonstrate the Comicraft application.

2. SUBMISSION CHECKLIST

S.No	Item to Submit	Status
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	[Yes / NA]
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	[ENTER URL / NA]
7	APK / Executable binary (if applicable)	NA
8	Dockerfile / Containerization configuration	[Yes / NA]
9	Test files and test results	[Yes / NA]
10	Demo video / walkthrough	[Yes / NA]

3. PROJECT DESCRIPTION

3.1 Overview

Comicraft is a digital comic creation platform developed to provide users with a simple and interactive environment for creating comics.

The application focuses on making the comic creation process easier by providing features such as creating comic content, organizing comic panels/pages, adding text and visual elements, editing content, and managing completed comic projects.

The main objective of Comicraft is to combine creativity and technology into a single platform where users can create and manage digital comic content without requiring advanced technical knowledge.

3.2 Objectives

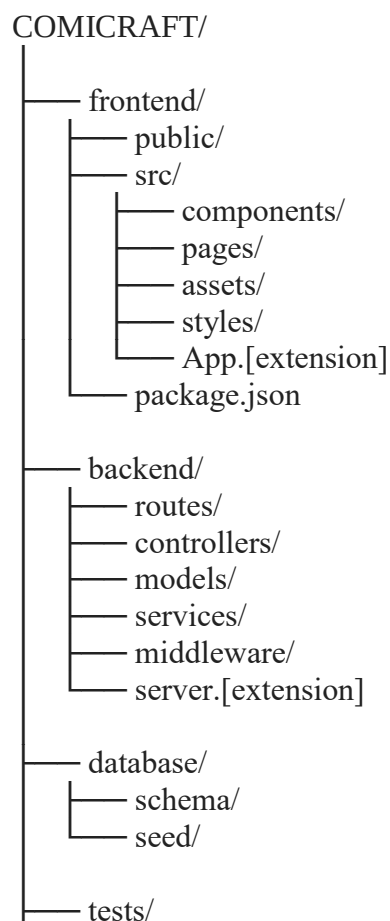
The major objectives of Comicraft are:

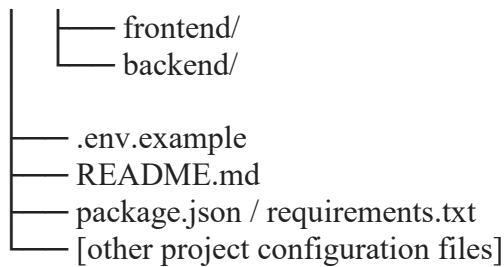
1. To provide an easy-to-use platform for creating digital comics.
2. To allow users to create and organize comic pages and panels.
3. To provide tools for adding and editing comic content.
4. To provide a structured interface for managing comic projects.
5. To reduce the complexity involved in creating digital comics.
6. To provide an interactive and user-friendly comic creation experience.
7. To enable users to save and manage their created comic projects.
8. To provide a platform that can be extended with additional comic creation features in the future.

4. FILE / FOLDER STRUCTURE

The Comicraft project is organized into separate folders and files to make the source code easy to understand, maintain, and deploy.

A typical project structure is shown below:





Note: The above structure should be adjusted to match the actual Comicraft project folders before submission.

5. DESCRIPTION OF IMPORTANT FILES AND FOLDERS

File / Folder | Description

"frontend/" | Contains the user interface and client-side functionality of Comicraft.
"backend/" | Contains server-side logic and APIs, if applicable.
"components/" | Contains reusable UI components used throughout the application.
"pages/" | Contains the major pages/screens of the Comicraft application.
"assets/" | Contains images, icons, fonts, and other static resources.
"routes/" | Contains API/application routes, if applicable.
"controllers/" | Handles application logic for backend requests, if applicable.
"models/" | Contains database models or data structures, if applicable.
"database/" | Contains database schema and seed information, if applicable.
"tests/" | Contains test cases and testing-related files.
".env.example" | Provides the required environment variables without exposing sensitive credentials.
"README.md" | Contains project information and instructions for installation and execution.
"package.json" / "requirements.txt" | Contains the dependencies required to run the project.

6. DEPENDENCIES AND REQUIREMENTS

The following software and tools may be required to run Comicraft locally:

- Operating System: Windows / Linux / macOS
- Modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox
- [Node.js version, if applicable]
- [Python version, if applicable]
- [Database name and version, if applicable]
- Internet connection, if external services are used
- Project dependencies listed in "package.json" / "requirements.txt"

All required project dependencies are provided through the project's dependency/configuration files.

7. ENVIRONMENT CONFIGURATION

Comicraft may require environment variables for application configuration.

A sample environment configuration file is provided as:

.env.example

The actual ".env" file should be created locally using the values required by the project.

Example:

```
DATABASE_URL=[ENTER DATABASE URL]
API_KEY=[ENTER API KEY IF REQUIRED]
PORT=[ENTER PORT]
OTHER_CONFIGURATION=[ENTER VALUE]
```

Sensitive credentials and private API keys should not be included directly in the submitted source code.

8. DEPLOYMENT / ACCESS DETAILS

The Comicraft application can be run locally using the instructions provided in this document and the project README file.

Field| Details

Hosted / Deployed URL| [ENTER DEPLOYED COMICRAFT URL]

Demo Username| [ENTER USERNAME / NA]

Demo Password| [ENTER PASSWORD / NA]

Platform / Hosting Provider| [ENTER HOSTING PROVIDER / LOCALHOST]

Repository Link| [ENTER GITHUB/GITLAB LINK]

Demo Video Link| [ENTER VIDEO LINK]

The demo credentials, if applicable, are provided for evaluator access to the Comicraft application.

9. DEPLOYMENT DETAILS

If the application is hosted online, the deployed version of Comicraft can be accessed through the hosted URL provided above.

The deployment configuration includes:

1. Uploading or connecting the Comicraft source repository to the hosting platform.
2. Installing all required project dependencies.
3. Configuring required environment variables.
4. Configuring the database, if applicable.
5. Building the application.
6. Starting the application using the configured start command.

7. Testing the deployed application through the provided URL.

If the project is being evaluated locally, the evaluator can follow the local execution instructions provided in Section 10.

10. RUN INSTRUCTIONS

Step 1: Download / Clone the Project

Download the Comicraft project files or clone the project repository.

```
git clone [ENTER REPOSITORY URL]
```

Move into the project directory:

```
cd Comicraft
```

Step 2: Install Dependencies

Install the required dependencies using the project's dependency file.

For a Node.js-based project:

```
npm install
```

For a Python-based project:

```
pip install -r requirements.txt
```

Use the command corresponding to the actual Comicraft implementation.

Step 3: Configure Environment Variables

Create a ".env" file in the required project directory.

Copy the contents of ".env.example" into ".env" and provide the required configuration values.

Example:

```
DATABASE_URL=[ENTER VALUE]  
API_KEY=[ENTER VALUE IF REQUIRED]  
PORT=[ENTER VALUE]
```

Step 4: Configure the Database

If Comicast uses a database, create/configure the database using the provided schema or database setup files.

Run the required database migration or seed commands.

Example:

[ENTER DATABASE SETUP COMMAND]

If no external database is required, this step can be marked as Not Applicable.

Step 5: Start the Application

For a Node.js application:

npm start

or, during development:

npm run dev

For a Python application:

python [ENTER MAIN FILE].py

Use the command defined in the actual Comicast project.

Step 6: Open the Application

After the application starts successfully, open the displayed local URL in a web browser.

Example:

http://localhost:[PORT]

The Comicast home page should be displayed.

11. BASIC APPLICATION WORKFLOW

The basic workflow of Comcraft is as follows:

1. Open the Comcraft application.
2. Access the main application/dashboard.
3. Create a new comic project.
4. Add comic pages or panels.
5. Add and edit comic content.
6. Customize the comic according to the user's requirements.
7. Save the comic project.
8. Review the completed comic.
9. Export, download, or share the comic if the corresponding feature is implemented.

12. TESTING

The Comcraft application should be tested to verify that the major features work correctly.

The following areas are considered during testing:

- Application startup
- User interface navigation
- Comic project creation
- Comic page/panel creation
- Adding and editing comic content
- Saving comic projects
- Loading previously saved content
- Input validation
- Database operations, if applicable
- API functionality, if applicable
- Responsive interface behavior
- Error handling
- Application deployment

Sample Test Cases

Test Case| Expected Result| Status

Open Comcraft application| Application loads successfully| Pass

Create a new comic| New comic project is created| Pass

Add comic panel/page| Panel/page is added successfully| Pass

Add text/content| Content appears correctly in the comic| Pass

Edit comic content| Changes are reflected correctly| Pass

Save comic| Comic/project is saved successfully| Pass

Open saved comic| Saved comic data is loaded correctly| Pass

Invalid input| Appropriate validation/error message is displayed| Pass

Refresh application| Application remains functional and data is preserved where applicable| Pass

Access deployed application| Application is accessible through the provided URL| [Pass/NA]

13. DEMO VIDEO / WALKTHROUGH

A demonstration video can be provided to show the main functionality of Comicraft.

The video demonstrates:

1. Opening the Comicraft application.
2. Navigating through the application.
3. Creating a comic project.
4. Adding comic pages/panels.
5. Editing comic content.
6. Saving the project.
7. Viewing the completed comic.
8. Demonstrating other major implemented features.

Demo Video: [PASTE DEMO VIDEO LINK]

14. KNOWN ISSUES / LIMITATIONS

S.No	Known Issue / Limitation	Workaround / Status
------	--------------------------	---------------------

- | | | |
|---|--|--|
| 1 | Some advanced comic customization features may be limited depending on the current implementation. Additional customization features can be added in future versions. | |
| 2 | Application functionality may depend on correct environment and dependency configuration. Follow the setup instructions and configure the ".env" file correctly. | |
| 3 | Features requiring external services may not work without valid API/service credentials. Configure the required API keys or external services before execution. | |
| 4 | Performance may depend on the user's device, browser, and available resources. Use a modern browser and adequate system resources. | |
| 5 | Some features may require an active internet connection. Ensure that the system has internet access where required. | |

15. FUTURE ENHANCEMENTS

The following features can be considered for future versions of Comicraft:

1. Advanced comic panel customization.
2. More character and background templates.
3. AI-assisted comic story generation.
4. AI-assisted dialogue and speech-bubble generation.
5. More export formats such as PDF and image formats.
6. Improved comic sharing functionality.
7. User profiles and cloud-based comic storage.
8. Collaboration between multiple users.
9. Additional themes, layouts, fonts, and visual effects.
10. Improved mobile responsiveness.
11. Version history for comic projects.
12. Enhanced search and organization of saved comics.

16. CONCLUSION

Comicraft provides a structured digital environment for creating and managing comics. The project combines an interactive user interface with the required application logic and supporting resources to simplify the comic creation process.

The submitted executable files, source code, dependency files, configuration files, and setup instructions are organized to allow the evaluator to run and verify the project.

The project demonstrates the practical application of software development concepts through the development of a creative and interactive comic creation platform.

17. EVALUATOR ACCESS INFORMATION

Project Name: Comicraft

Repository: [PASTE REPOSITORY LINK]

Live Application: [PASTE DEPLOYED URL / NA]

Demo Username: [ENTER / NA]

Demo Password: [ENTER / NA]

Demo Video: [PASTE VIDEO LINK / NA]

Local Run Command: [ENTER ACTUAL COMMAND]

Project Status: Ready for evaluation